

**TP4 – POO en Python**

**Pr. Houda BENHAR**

**Exercice 1.**

```
class Compte:
    def __init__(self, nom, numero, balance):
        self.nom = nom
        self.numero = numero
        self.balance = balance

    def deposer(self, montant):
        self.balance += montant

    def retirer(self, montant):
        self.balance -= montant

compte_john = Compte(nom="Asmaa", numero="12345", balance=20000)
compte_john.deposer(montant=3000)
compte_john.retirer(montant=200)
```

**Exercice 2.**

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade #0 - 100
    def get_grade(self):
        return self.grade

class Course:
    def __init__(self, name, max_students, students = []):
        self.name = name
```

```
self.max_students = max_students
self.students = students
```

```
def add_student(self, student):
    if len(self.students) < self.max_students:
        self.students.append(student)
        return True
    return False
```

```
def get_average_grade(self):
    value = 0
    for student in self.students:
        value += student.get_grade()
    return value / len(self.students)
```

```
s1 = Student ("Ibrahim", 19, 95)
s2 = Student("Fatima", 19, 75)
s3 = Student ("Bilal", 19, 65)
```

```
course = Course("Science", 2)
course.add_student(s1)
course.add_student(s2)
print(course.get_average_grade())
```

### **Exercise 3.**

```
class myString:
    def __init__(self, s):
        self.s = s

    def append(self, x):
        self.s = self.s + x
        return self.s

    def pop(self, i):
        s1 = self.s[0:i]
        s2 = self.s[i + 1:len(self.s)]
```

```
return s1 + s2
```

```
def modifier(self, i):  
    pass
```

*# Tester la classe*

```
s1 = myString("Hello")  
s2 = myString("bonjour")  
print(s1.append(" world !")) # affiche 'Hello world !'  
print(s2.pop(2)) # affiche 'bojour'.
```

#### **Exercice 4.**

```
class Person():  
    def __init__(self, name, age):  
        self._name = name  
        self._age = age  
  
    def __str__(self):  
        return f'Name: {self._name}, Age: {self._age}'  
  
class Employee(Person):  
    def __init__(self, name, age, salary):  
        super().__init__(name, age)  
        self._salary = salary  
  
    def show_salary(self):  
        print("Salary of this employee is: ", self._salary)  
  
class Manager(Employee):  
    def show_salary(self):  
        print("Salary of this manager is: ", self._salary)  
  
p = Person("Ali", 30)  
e = Employee("Israe", 25, 5000)
```

```
m = Manager("Alae", 60, 20000)
```

```
print(p)
print(e)
m.show_salary()
e.show_salary()
```

### **Exercice 5.**

```
class Etudiant:
```

```
    repertoire = []
```

```
    def __init__(self, nom, prenom):
        self.prenom = prenom
        self.nom = nom
        self.uid = len(Etudiant.repertoire) + 1
        Etudiant.repertoire.append(self)
```

```
    def __str__(self):
        return "{} {}".format(self.prenom, self.nom)
```

```
ahmed = Etudiant("Ahmed", "Benbrahim")
```

```
amina = Etudiant("Amina", "Haddad")
```

```
omar = Etudiant("Omar", "Benjelloun")
```

```
for etudiant in Etudiant.repertoire:
```

```
    print(etudiant)
```

*# ou*

```
class Etudiant:
```

```
    repertoire = []
```

```
    def __init__(self, nom, prenom):
        self.prenom = prenom
        self.nom = nom
        self.uid = len(Etudiant.repertoire) + 1
```

```
def __str__(self):  
    return "{} {}".format(self.prenom, self.nom)
```

```
@classmethod  
def add_student(cls, std):  
    cls.repertoire.append(std)
```

```
ahmed = Etudiant("Ahmed", "Benbrahim")  
amina = Etudiant("Amina", "Haddad")  
omar = Etudiant("Omar", "Benjelloun")  
Etudiant.add_student(ahmed)  
Etudiant.add_student(amina)  
Etudiant.add_student(omar)
```

```
for etudiant in Etudiant.repertoire:  
    print(etudiant)
```

### **Exercice 6.**

```
class Forme:  
    def __init__(self, largeur, longueur):  
        self.__largeur = largeur  
        self.__longueur = longueur  
  
    def get_largeur(self):  
        return self.__largeur  
  
    def set_largeur(self, largeur):  
        self.__largeur = largeur  
  
    def get_longueur(self):  
        return self.__longueur  
  
    def set_longueur(self, longueur):  
        self.__longueur = longueur
```

```
class Triangle(Forme):
    def aire(self):
        return self.get_largeur() * self.get_longueur() / 2
```

```
class Rectangle(Forme):
    def aire(self):
        return self.get_largeur() * self.get_longueur()
```

```
class Cube(Forme):
    def __init__(self, largeur, longueur, hauteur):
        super().__init__(largeur, longueur)
        self.__hauteur = hauteur

    def get_hauteur(self):
        return self.__hauteur

    def set_hauteur(self, hauteur):
        self.__hauteur = hauteur

    def volume(self):
        return self.get_largeur() * self.get_longueur() * self.get_hauteur()
```

*# Programme principal*

```
triangle = Triangle(3, 4)
rectangle = Rectangle(5, 6)
cube = Cube(2, 3, 4)
```

```
print("Aire du triangle:", triangle.aire())
print("Aire du rectangle:", rectangle.aire())
print("Volume du cube:", cube.volume())
```